

# 2025 Fall CSED451 Assignment2 Report

Team Name: openGameLab

Team Member #1: 안재영

컴퓨터공학과/20220019/enter21

Team Member #2: 정윤희

컴퓨터공학과/20240385/jyhyeok

## 1. Technical Details

개발 환경: Visual Studio 2022 버전 17.14.9 (July 2025)

cpp 확장자 파일을 powershell 스크립트 파일로 만든 후 powershell에서 빌드하여 구현 결과를 확인할 수 있다. 또한, OpenGL을 사용하는 코딩에 익숙해야 한다.

## 2. Implementation Details

### 2-1. Outline

이번 과제의 구현은 이전의 핵심 로직들을 유지하면서 새로운 목표들을 구현하기 위해 적과 플레이어에 수정을 거치는 방식으로 작성되었다. 플레이어의 경우, 남은 목숨의 개수를 표현하는 orbiting object를 그리기 위해 drawPlayerOrbitingEntities라는 함수를 구현했다. 이 함수는 기존에 존재하는 drawCircle 함수를 이용하며, 최종적으로는 display 함수에서 호출되어 사용된다.

적의 경우 플레이어보다 더 많은 변경사항이 필요하였는데, 우선 여러 명의 적을 효율적으로 관리할 수 있도록 적의 구현 방식이 구조체로 변경되었다. 기존에는 하나의 적만 존재했기에 적에 관한 변수들을 전역 변수로 두고 사용하고 있었는데, 각각의 적이 서로 다른 특성을 가질 수 있도록 구조체 내에서 관리하도록 구현하였다. 또한, 적의 하강 모션이나 HP 기반 모델링 변화, 플레이어 추적 회전, 계층적 다리 애니메이션을 구현하기 위해 새로운 함수들이 구조체의 내부에 새롭게 작성되었다.

## 2-2. Design Rationale (Including Additional Requirements)

이번 프로젝트에서는 크게 적 오브젝트의 구조적 확장과 시각적 다양성을 목표로 하였다. 기존 구현은 단일 객체 중심으로 되어 있어 여러 적을 다른 특징을 가지도록 생성하고 동시에 관리하기 어려웠고, 모델 역시 고정된 위치에서 움직이지 않아 게임 장면의 몰입감이 떨어지는 한계가 있었다. 이에 아래와 같은 방식으로 적을 개선하였다.

- 1) 효율적 객체 관리: 다수의 적을 일괄적으로 관리할 수 있도록 구조체 기반의 데이터 모델로 통합하고, 개별 적의 위치, 속도, 체력, 회전 각도 등 상태를 구조 화하여 코드의 유지보수성과 확장성을 높인다.
- 2) 동적 연출 및 상호작용성 향상: 적이 단순히 화면의 고정된 위치에 존재하는 것이 아니라 시간이 지나면서 하강하도록 구현하였다. 또한 추가적으로 플레이어 방향을 바라보는 움직임을 부여(Additional 1)하여 게임의 긴장감을 높인다.
- 3) 시각적 다양성과 생동감 표현: 체력 변화에 따라 적의 원형 부분이 줄어들면서 가시가 드러나며 색이 붉어지도록 하였고(Additional 2), 심장이 박동하듯 주기적으로 변하는 적의 반지름이나 계층적 변환이 적용된 꼬리 오브젝트(Additional 3)를 통해 생동감을 느낄 수 있도록 하였다.

또한 플레이어의 목숨 수를 직관적으로 확인할 수 있도록, 주변에 남은 목숨 수만큼의 회전하는 원형 오브젝트를 추가하였다.

## 2-3. Design Implementation

적의 구조체화는 struct Enemy의 형태로 정의하고, 위치(x, y)와 속도, 체력, 크기, 각도 등 렌더링 및 동작에 필요한 모든 상태 변수를 구조체 내부에 포함시켜 개별적인 값들을 활용하도록 하였다. 다수의 적을 관리할 수 있도록 std::vector<Enemy> 컨테이너를 도입하여, 각 프레임마다 적에 관한 상호작용이 필요할 경우 반복문을 통해 모든 적에 순서대로 접근하고 처리할 수 있도록 구현하였다. 이로써 적의 생성, 삭제, 충돌 처리 등을 효과적으로 확장할 수 있게 되었다.

적의 연출 추가는 우선 update 단계에 vector에 담긴 각 적 엔티티에 접근하면서 y 좌표에 fallSpeed \* deltaTime 만큼의 변화를 주도록 하였다. 또한 매 프레임마다 플레이어와 적의 상대 위치를 벡터로 계산하고, atan2() 함수를 사용해 두 점을 잇는 방향각을 구한 후 적 렌더링 시 좌표계에 해당 각도만큼의 glRotate를 가하여 적이 항상 플레이어를 바라보고 있도록 구현하였다. 이로써 적들의 움직임에 플레이어 중심적 상호작용을 적용

하게 되었다.

기존에 적의 체력을 상단의 체력 바로 구현하던 것에도 변화가 이루어졌다. 체력 바의 길이를 계산하던 것과 동일한 방식의 비율 계산 방식으로 수치와 색을 조절하기 위해 함수 `lerp`와 `lerpColor`를 작성하였다. 두 함수를 사용해 `bodyScale`, `tailLength` 등의 파라미터에 곱해 몸통이 갈수록 작아지도록, 색깔에 적용해 갈수록 붉어지도록 구현하였다.

지속적인 적의 움직임은 `update`에서 `tailSpeed * dt`로 계속 증가하는 `tailPhase`를 매개변수로 한 삼각함수를 통해 주기적으로 구현된다. `drawBoss` 함수 내에서는 `bodyScale`에 곱해지는 `pulse`에 사용되어 적 엔티티가 마치 심장이 두근대는 듯한 생동감을 가지도록 한다. 거기에 추가된 몸통 하단의 꼬리 오브젝트는 두 개의 파츠가 계층적으로 연결되어 있다. 몸통의 변환을 기준으로 `glPushMatrix`와 `glPopMatrix`를 이용해 해당 좌표계에서 `translate` 및 `tailPhase`의 삼각함수를 사용한 `rotation`으로 흔들림 효과를 연출하였다.

플레이어의 경우 우선 엔티티들의 회전을 관리하기 위해 `orbitangle`이라는 변수를 선언한다. `drawPlayerOrbitingEntities` 함수는 플레이어가 살아있지 않으면 즉시 `return`되어 그림을 그리지 않는다. 엔티티의 반지름, 회전 반지름을 변수로 선언해준다. 플레이어의 현재 `life`만큼 `for`문을 실행한다. `for`문 안에서는 엔티티 사이 각도를 `angle`이라는 변수에, 엔티티의 중점을 `ex`, `ey`라는 변수에 현재 플레이어 위치, 엔티티 반지름, 그리고 `angle` 변수를 이용하여 연산한 후 저장한다. 그 후 지금까지 `draw`함수들을 구현했던 것처럼 OpenGL 함수들을 이용하여 화면에 그린다.

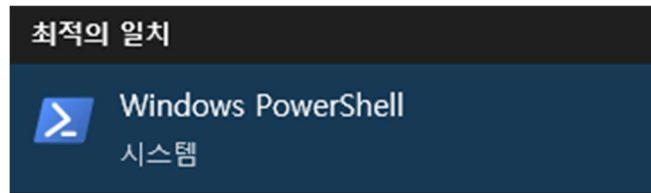
## 2-4. Additonal Background

본 구현을 위해서는 OpenGL의 행렬 변환 및 좌표계, 기본적인 삼각함수에 대한 이해가 필요했다. 구체적인 배경 지식들은 아래와 같다.

- 1) `glPushMatrix`와 `glPopMatrix`: 각각 현재의 모델뷰 행렬을 스택에 저장하고 복원하는 함수로, 이를 통해 계층적으로 구현된 모델링들의 변환을 올바르게 관리할 수 있다.
- 2) 삼각함수: 본 과제의 구현에서 꼬리의 움직임, 적 오브젝트 크기의 박동 등은 `sin` 함수를 사용해 일정 범위 내에서 주기적으로 움직이도록 구현되었고, 적과 플레이어 사이의 방향 벡터를 구하는 데에는 `atan2` 함수가 사용되었다.

### 3. End-User Guide

Windows powershell을 실행한다.



프로젝트가 저장된 파일 위치로 이동한다.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

새로운 크로스 플랫폼 PowerShell 사용 https://aka.ms/pscore6

PS C:\Users\Jin_Note> cd CSED451
PS C:\Users\Jin_Note\CSED451> cd assn1
PS C:\Users\Jin_Note\CSED451\assn1> _
```

.\build.ps1을 입력한다.

```
PS C:\Users\Jin_Note\CSED451\assn1> .\build.ps1

디렉터리: C:\Users\Jin_Note\CSED451\assn1

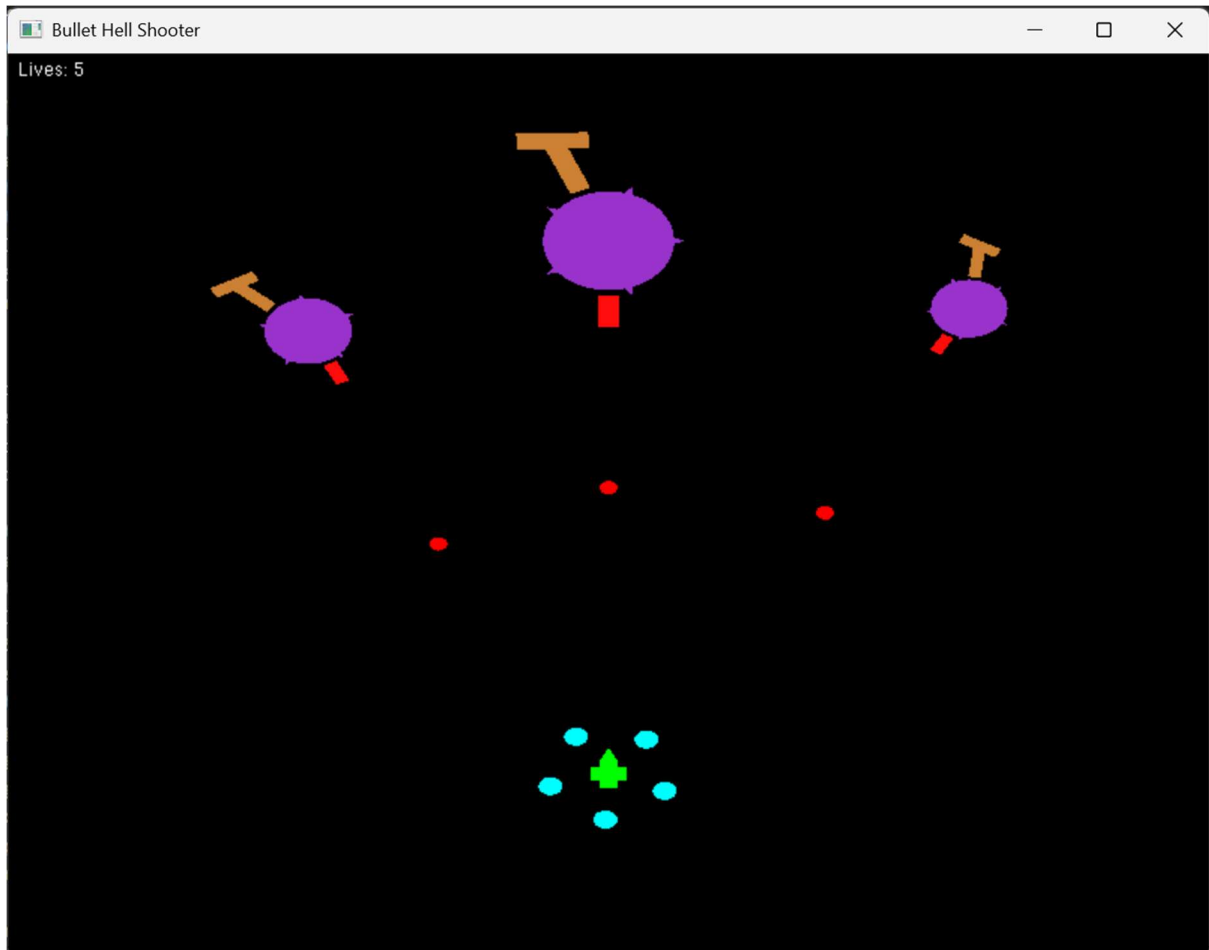
Mode                LastWriteTime         Length Name
----                -
d-----         2025-10-13 오전 11:31             build

*****
** Visual Studio 2022 Developer PowerShell v17.14.9
** Copyright (c) 2025 Microsoft Corporation
*****
Microsoft (R) C/C++ 최적화 컴파일러 버전 19.44.35213(x64)
Copyright (c) Microsoft Corporation. All rights reserved.

assn1.cpp
Microsoft (R) Incremental Linker Version 14.44.35213.0
Copyright (c) Microsoft Corporation. All rights reserved.

/out:C:\Users\Jin_Note\CSED451\assn1\build\build.exe
/LIBPATH:C:\Users\Jin_Note\CSED451\assn1\lib
freeglut.lib
glew32.lib
opengl32.lib
C:\Users\Jin_Note\CSED451\assn1\build\assn1.obj
Execution file build successfully. Running...
```

잘 실행되는 것을 확인할 수 있다.



프로그램 종료 시 powershell 창에는 아래 메시지가 출력된다.

```
Execution exited.

PS C:\Users\Jin_Note\CSED451\Wassn1>
```

이후 다시 빌드하여 프로그램을 실행시키는 것 또한 가능하다.

만약 Powershell 스크립트 실행 정책이 제한되어 있는 등의 이유로 Powershell에서 스크립트를 실행할 수 없다면, Get-ExecutionPolicy 명령어를 입력하여 실행 정책이 제한되어 있는지 확인해야 한다. 만약 그렇다면, Set-ExecutionPolicy RemoteSigned -Scope CurrentUser 명령어를 입력하여 스크립트 실행 정책을 변경해야 한다. 정책 여부 변경을 묻는 메시지가 나타나면 Y를 입력하여 스크립트 실행 정책을 변경하면 실행이 잘 된다.

```
PS C:\Users\Jin_Note> Set-ExecutionPolicy RemoteSigned -Scope CurrentUser

실행 규칙 변경
실행 정책은 신뢰하지 않는 스크립트로부터 사용자를 보호합니다. 실행 정책을 변경하면 about_Execution_Policies 도움말
항목 (https://go.microsoft.com/fwlink/?LinkID=135170)에 설명된 보안 위험에 노출될 수 있습니다. 실행 정책을
변경하시겠습니까?
[Y] 예(Y) [A] 모두 예(A) [N] 아니요(N) [L] 모두 아니요(L) [S] 일시 중단(S) [?] 도움말 (기본값은 "N") : Y
PS C:\Users\Jin_Note>
```

## 4. Discussions/Conclusions

### 4-1. Obstacles

이번 과제에서 가장 큰 어려움은 계층적 변환을 구현하기 위한 `glPushMatrix`와 `glPopMatrix`의 사용이었다. 적의 각 구성 요소나 꼬리와 같이 새로운 좌표계를 저장 및 복원할 필요가 있을 때마다 해당 함수를 쌍으로 호출해야 했는데, 행렬 스택의 상태를 잘못 관리하면 엔티티가 월드 좌표계의 중심에 잘못 위치하는 경우가 빈번히 발생했다. 또한 각 시점에 좌표계의 상태를 정확히 파악하고 `translate`의 거리와 `rotation`의 각도를 적절하게 설정하는 것도 여러 번의 시행착오를 요구하였다.

### 4-2. Improvement Idea

현재 구현된 적의 움직임은 시각적으로는 충분히 생동감이 개선되었으나 움직임 자체는 상당히 단순한 상태이다. 따라서 적의 공격 패턴이나 움직임 패턴 등을 더 다양하게 설계한다면 게임성을 높일 수 있다.

또한, 현재 게임의 여러 주기적인 움직임이 한 가지 변수인 `tailPhase`의 `sin` 함수로 구현되어 있는데, 더 다양한 변수와 함수를 조합하여 각 적마다 다채로운 움직임을 가지도록 한다면 게임의 생동성을 더 높일 수 있을 것이다.

### 4-3. Lessons

이번 과제를 통해 OpenGL의 행렬 변환 구조와 계층적 애니메이션 구현의 복잡성을 깊이 체감하고 이해할 수 있었다. 특히 행렬 스택의 상태를 정확하게 유지하는 것이 얼마나 중요한지 여러 번의 시행착오를 통해 깨닫게 되었다.

또한 여러 적을 구조체 및 벡터 컨테이너로 관리하면서 객체 단위의 구현과 관리가 가진 확장성과 유지보수의 장점을 이해하게 되었다.

## 5. References

Learn OpenGL - <https://learnopengl.com/>

## 6. AI-assisted Coding References

이번 과제의 약 40%가 AI(ChatGPT)의 도움을 받아 구현되었다. 이전에 존재하던 코드의 전체 게임 시스템, 기본 도형 그림 함수 등은 그대로 유지한 채 적의 경우 관련 코드들을 구조체화하고, 그래픽을 그리는 부분에 여러 계층 구조를 작성하는 것이 본 과제의 전반적인 작업이었다. 이 중 AI의 도움을 받은 것은 삼각함수를 사용해 꼬리 및 적 크기의 주기적인 움직임을 구현하는 방법과 적 모델링에 뾰족한 내부 스파이크를 추가하는 방법이었다.

## 7. individual contribution

이번 프로젝트는 두 팀원이 50:50 비율로 각자의 역할을 나누어 충실히 수행하였다. 각 지난 코드들의 리팩토링과 플레이어 추가 기능 구현, 적의 구조 변경 및 추가 기능 구현으로 구현 파트를 나누고 각각의 작업 내용에 대한 보고서를 작성하여 프로젝트를 성공적으로 완수하였다.